QUANTUM SKY

PROJECT 137

Application Development Assignment

Project: Build a small, working application with the AI tools you normally use, then walk us through **how you built it**.

We expect you to use AI tools, and we expect them to write much of the code. What we want to see is the engineering around that: how you structured and designed the application, how you set up and steered your tools, how you checked their work, and what you added that a first internal release needs. Keep it small. We scoped it to a few hours on purpose.

The scenario: A network operations team runs a European backbone that connects regional operations sites to two data centers. Before the next planning cycle they want answers to three questions:

1. Where does the network have single points of failure?
2. Which links does the traffic depend on most?
3. What happens when a given link or site goes down?

Today they work from static diagrams. You build them a first tool.

Two inputs. You need both:

- **(A) The backbone:** the public research network **nobel-eu** from SNDlib, the Survivable Network Design Library. Download https://sndlib.put.poznan.pl/download/sndlib-networks-xml/nobel-eu.xml (or the same data as text: https://sndlib.put.poznan.pl/download/sndlib-networks-native/nobel-eu.txt). It has 28 European cities with coordinates, 41 links and 378 traffic demands. Please cite SNDlib in your README.
- **(B) The sites:** the table below. We made it up for this exercise; save it as sites.csv. Each row adds one node. Its attaches_to values are node ids: a city id from file A (such as Hamburg) or another row's site_id. Each value is one access link with the row's capacity. For example, the first row adds DC-NORTH with two links of capacity 200, one to Hamburg and one to Copenhagen.

site_id,site_type,attaches_to,access_capacity

DC-NORTH,data-center,Hamburg;Copenhagen,200

DC-SOUTH,data-center,Milan;Zurich,200

OPS-WEST,operations-site,Dublin,40

OPS-CENTRAL,operations-site,Paris;Brussels,40

OPS-SOUTHEAST,operations-site,Athens,40

HUB-EAST,aggregation-hub,Warsaw,80

OPS-EAST-1,operations-site,HUB-EAST,40

OPS-EAST-2,operations-site,HUB-EAST;Budapest,40

OPS-IBERIA,operations-site,Madrid;Barcelona,40

Requirements:

Data

- **R1.** The application shall load inputs A and B into one network model. Merged, it has 37 nodes, 55 links and 391 demands (2,068 units); use this to check your merge.
- **R2.** The application shall reject bad input with an error that names the file and the row.
- **R3.** Every backbone link shall have a capacity of 550 units; the capacity modules in file A shall be ignored. Links and demands have no direction.
- **R4.** In addition to file A's demands, each of the six operations sites shall send 10 units to each data center, and the two data centers shall exchange 50 units.

Behavior

- **R5.** The application shall route each demand over the path or paths with the fewest hops. Where several such paths exist, it shall split the demand equally among them.
- **R6.** Operations sites and data centers shall not carry traffic between other nodes. Only backbone cities and the aggregation hub shall forward traffic.
- **R7.** A node or link shall be reported as critical if losing it (a) splits the network, (b) leaves any demand undeliverable, or (c) pushes any link above 100% utilization.
- **R8.** For any single failed node or link, the application shall report the demands that can no longer be delivered and every link above 100% utilization. Demands that start or end at a failed node shall be reported separately.

View

- **R9.** The application shall show the network on a map laid out from the coordinates, with each site near what it attaches to.
- **R10.** The view shall show the critical nodes and links and the five most utilized links, without relying on colour alone.
- **R11.** The user shall be able to fail and restore any node or link and see the R8 result without reloading the page.

Technology: The back end is Python or Go. The front end is TypeScript; React is welcome but not required. Any libraries. No database is needed. It shall start with one documented command.

What to send us: the repository, as a link we can read or a zip file, containing:

1. **the code, with its full git history.** Do not squash it; we read it.
2. **your AI instruction files**, such as CLAUDE.md or AGENTS.md, and any plan or specification the tool produced and you edited;
3. **README.md:** how to run it, and what is not production-ready;
4. **a slide deck** (PowerPoint or PDF, about 10 slides) that you will present in the follow-up session. Your audience is the panel: an operations lead and engineers. Cover, in an order you choose:
    - *Findings:* the answers to the team's three questions, with numbers, for the operations lead;
    - *Structure and design:* how the code is organized and why; the data model; the interface between back end and front end; which of these you decided and which you accepted from the tool;
    - *Setting up the AI:* what you told the tool before it wrote code, and which checks it had to pass (tests, type checks, linters, hooks);
    - *Verification:* how you know the numbers are right, other than the tool saying so. Which tests did you write or specify yourself, and where did their expected values come from?
    - *Production concerns:* for each of authentication and access control, logging, input validation and error handling, API versioning, configuration and secrets, and dependency and security checks: did you implement it? If not, why not, and what would you do for a first internal release? We prefer two done well to six done badly;
    - *Steering:* three to five moments where you stopped, redirected or overrode the tool. For each: what it produced, what you changed, and why;
    - *Unclear:* anything in this brief you found unclear, and how you resolved it;
    - *Time spent.*

The follow-up session (60 minutes). About 15 minutes for you to present your deck, including a short live demonstration of the application. Please keep to time. About 25 minutes of questions, most of them about your deck and your git history. Then about 15 minutes on a small change we ask for, made live with your own tools. Time for your questions at the end.

Rules:

- Use AI tools: Claude Code, Cursor, Copilot, whatever is in your workflow. You must be able to explain every meaningful decision and walk us through how any piece was built.
- Use only public data and open-source tools. Do not use code or documents that belong to a current or former employer.
- Do not put credentials, keys or tokens in the repository.

What we look at: How you structured and designed the application; how you set up, steered and checked your AI tools; which production concerns you handled and how you reasoned about the rest; whether the application meets the requirements; your git history; and how well you present your work to a mixed audience and answer questions.